

Reviewer Pack — Jacobi Recurrence b_2 Floor Constrains SoS-2 Max-CUT Ratio

Entry 6a8b922a8da3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 01:52:07 UTC

Jacobi Recurrence b_2 Floor Constrains SoS-2 Max-CUT Ratio

Entry ID: 6a8b922a8da3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 01:52:07 UTC

1. Conjecture

Field A (mathematical branch): Markov-Krein moment theory: Jacobi (three-term recurrence) parameters of empirical spectral measures via the Chebyshev/Wheeler algorithm **Field B** (complexity object): Sum-of-Squares degree-2 (Goemans-Williamson SDP) Max-CUT integrality ratio on 3-regular graphs

Statement:

For a connected 3-regular graph G on n vertices with adjacency matrix A , let $\mu_G = (1/n) \sum \delta_{\lambda_i(A)}$ be the empirical spectral measure and let $\{a_k, b_k\}_{k \geq 0}$ be its Jacobi parameters (so the orthonormal polynomials satisfy $x p_k = b_k p_{k+1} + a_k p_k + b_{k-1} p_{k-1}$, $b_k > 0$). Define the spectral-SDP ratio $\rho(G) := \text{MaxCUT}(G) / [(n/4)(3 - \lambda_{\min}(A))]$ and the Jacobi defect $J(G) := b_2(G) - \sqrt{2}$ (where $\sqrt{2}$ is the Kesten-McKay limit for 3-regular trees). Conjecture: there is an absolute constant $c \leq 4$ such that for every 3-regular G on $n \geq 8$ vertices, $\rho(G) \geq 0.879 \implies J(G) \geq -c \cdot \log(n)/\sqrt{n}$; a single instance with $\rho(G) \geq 0.879$ and $b_2(G) < \sqrt{2} - 4 \cdot \log(n)/\sqrt{n}$ refutes it.

Rationale (proposer's reasoning):

Jacobi parameters reduce a spectral measure to its scattering data, and the Kesten-McKay law (the universal local limit for random 3-regular spectra) pins $\{a_k=0, b_0=\sqrt{3}, b_k=\sqrt{2}$ for $k \geq 1\}$; deviations of b_2 measure how far G 's local spectrum departs from the free-probabilistic ideal. SoS-2 for Max-CUT is exactly the GW SDP whose tightness for 3-regular graphs is governed by spectral concentration around Kesten-McKay, so a downward push on b_2 should signal exactly the moment-matrix obstruction that lets the SDP overshoot the integral cut. This bridges Markov-Krein scattering data (rare in complexity theory, <5 papers crossed with SDP relaxations) to the open SoS-2 integrality-gap question via a quantity computable in $\text{poly}(n)$ without any SDP solver.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ddd419ec3cde2f82

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{10, 14, 18, 20\}$ with 30 seeds each (120 instances), sample random 3-regular graphs; compute $\rho(G)$ and $J(G) = b_2 - \sqrt{2}$. Conjecture is SUPPORTED iff $\geq 27/30$ per n satisfy $\rho \geq 0.879 \implies J \geq -4 \cdot \log(n)/\sqrt{n}$, with no single instance violating $J < -4 \cdot \log(n)/\sqrt{n}$ while $\rho \geq 0.879$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Jacobi recurrence parameters empirical spectral measure 3-regular graph Kesten-McKay - Goemans-Williamson Max-CUT integrality ratio cubic graph spectral SDP 0.878 - moment method Chebyshev Wheeler algorithm orthogonal polynomials Max-CUT semidefinite relaxation

Top relevant hits considered: - [s2:2512.04951] MAX BISECTION might be harder to approximate than MAX CUT - [s2:10.1145/380752.380837] On the integrality ratio of semidefinite relaxations of MAX CUT

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def lanczos(A, v0, k):
    n = len(A)
    v = [v0]
    H = []
    for _ in range(k):
        w = A @ v[-1]
        alpha = sum(v[-1][i] * w[i] for i in range(n))
        w -= alpha * v[-1]
        beta = math.sqrt(sum(w[i] ** 2 for i in range(n)))
        if beta == 0:
            break
        v.append(w / beta)
        H.append([alpha, beta])
    return v, H

def chebyshev_recurrence(A, b, k):
    n = len(A)
```

```

sigma = [[0] * (k + 2) for _ in range(k + 1)]
sigma[0][0] = 1
sigma[1][1] = 1
for i in range(2, k + 1):
    sigma[i][i] = 1
    for j in range(i - 1, -1, -1):
        sigma[j][i] = (A @ sigma[j][i - 1]) / b[i]
        sigma[j][i - 1] -= A @ sigma[j + 1][i - 1]
return sigma

def max_cut(G):
    n = len(G)
    best_cut = 0
    for i in range(1 << n):
        cut_size = sum((i >> j) & 1 for j in range(n))
        if cut_size > n // 2:
            continue
        cut_value = sum(G[i][j] * ((i >> j) & 1) * ((i >> k) & 1) for j in range(n) for k in range(n))
        best_cut = max(best_cut, cut_value)
    return best_cut

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([10, 14, 18, 20])
    G = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0
        for j in range(i + 1, n):
            if random.random() < 3 / (2 * n - 4):
                G[i][j] = G[j][i] = 1

    v0 = [random.random() for _ in range(n)]
    v, H = lanczos(G, v0, 12)
    moments = [sum(v[-1][i] ** j for i in range(n)) / n for j in range(13)]

    sigma = chebyshev_recurrence(G, [math.sqrt(moments[i]) for i in range(1, 6)], 5)
    a = [sigma[0][i] - sigma[1][i] * sigma[2][i] / sigma[1][1] for i in range(3)]
    b = [math.sqrt(sigma[i + 1][i + 1] / sigma[i][i]) for i in range(5)]

    max_cut_value = max_cut(G)
    rho_G = max_cut_value / ((n / 4) * (3 - moments[2]))
    J_G = b[2] - math.sqrt(2)

    conjecture_holds = True
    counterexample = ""
    if rho_G >= 0.879 and J_G < -4 * math.log(n) / math.sqrt(n):
        conjecture_holds = False
        counterexample = f"p(G)={rho_G}, J(G)={J_G}, n={n}"

    return {
        "metric_name": "Jacobi Defect",
        "metric_value": J_G,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\"}, \"metric_

    mean_J = sum(r["metric_value"] for r in results) / len(results)
    std_J = math.sqrt(sum((r["metric_value"] - mean_J) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_J} std={std_J} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print("RESULT: FALSIFIED counterexample=\"\" first_failing_seed=NA")
    else:
        print(f"RESULT: INCONCLUSIVE not enough seeds tested or no clear outcome")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9752c379.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9752c379.py", line 67, in run_trial
    v, H = lanczos(G, v0, 12)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9752c379.py", line 23, in lanczos
    w = A @ v[-1]
          ^~~~~
TypeError: unsupported operand type(s) for @: 'list' and 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the `lanczos` routine (using Python lists instead of numpy arrays for matrix-vector multiplication), so no data was produced and the pre-registered support condition cannot be evaluated. | next: Fix the `lanczos` implementation to use numpy arrays (`np.asarray` on `v0` and `A`, or store `v` as an `ndarray`) and rerun the 120-instance protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	239893
2	preregistration	claude_max	opus	0	0	7648
3	novelty	claude_max	opus	0	0	5568
4	novelty	claude_max	opus	0	0	8431
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	126407
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	191449
7	judge	claude_max	opus	0	0	4564

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 583961 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/6a8b922a8da3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6a8b922a8da3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6a8b922a8da3.tar.gz` (if generated)